微算機系統實習

第12組專案報告

LAB 05

組別： 12
班級、姓名與學號：
資工三 112590450 李馥亘
資工三 112590452 林希芸
資工三 112590454 林妍蓁
資工三 112590455 吳哲丞
日期：2025.11.20

1. 實驗內容：

- 練習撰寫可於 Linux 核心載入與移除的驅動模組
- 熟悉模組的編譯、掛載（insmod）與卸載（rmmod）流程
- 撰寫 **Hello 模組** 驗證基本驅動架構
- 實作 **字元裝置驅動 (Character Device Driver)**，並實現 open、read、write、release、ioctl 等功能
- 撰寫應用層測試程式 test.c 驗證驅動與使用者層之間的互動

2. 實驗過程及結果

實驗過程

- Hello 模組撰寫
  - 撰寫 hellod.c 驅動程式，在模組載入（insmod）與移除（rmmod）時分別輸出訊息
  - 成功在 TX2 與虛擬機上編譯與執行，於 dmesg 觀察到“Hello”與“Goodbye”訊息
- 字元驅動模組撰寫
  - 撰寫 demo.c，以主設備號 60 建立字元裝置 /dev/demo
  - 實作 open、read、write、release、ioctl 等函式，並於 kernel log 印出操作資訊
  - 使用 register_chrdev() 向系統註冊裝置，unregister_chrdev() 於移除時釋放
- 應用層測試
  - 撰寫 test.c 程式，使用 fopen("/dev/demo", "w+") 開啟裝置後進行讀寫測試
  - 驗證應用程式與驅動間的溝通是否正確
- 編譯與載入
  - 使用 Makefile 編譯驅動模組：

    ```
    make        sudo insmod demo.ko        sudo rmmod demo        dmesg | tail
    ```

  - 確認模組訊息正常輸出，字元驅動能正確開啟、寫入與關閉

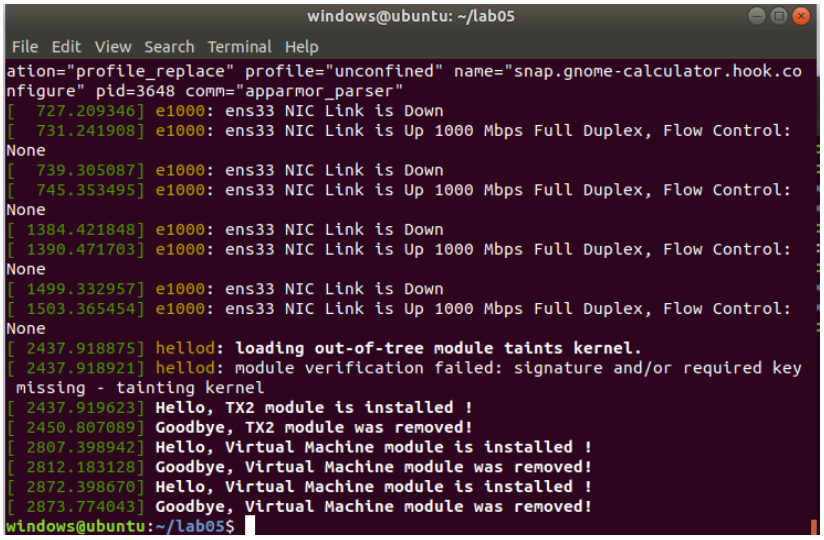
預期實驗結果

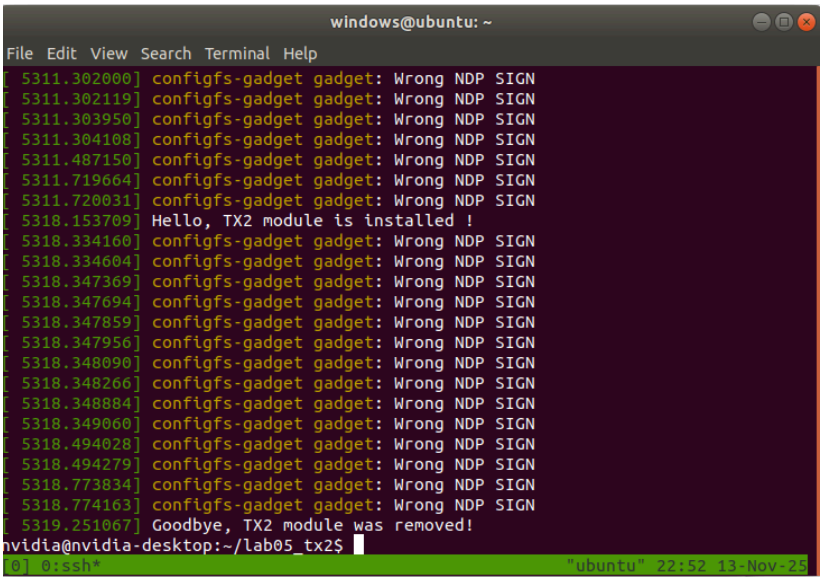
- 成功載入與移除 Hello 模組，並於 dmesg 顯示對應訊息
- /dev/demo 裝置可正常開啟、讀寫、釋放
- test.c 能成功與驅動程式互動並觀察到 kernel log 輸出
- 驅動模組能穩定運行於虛擬機與 TX2 上

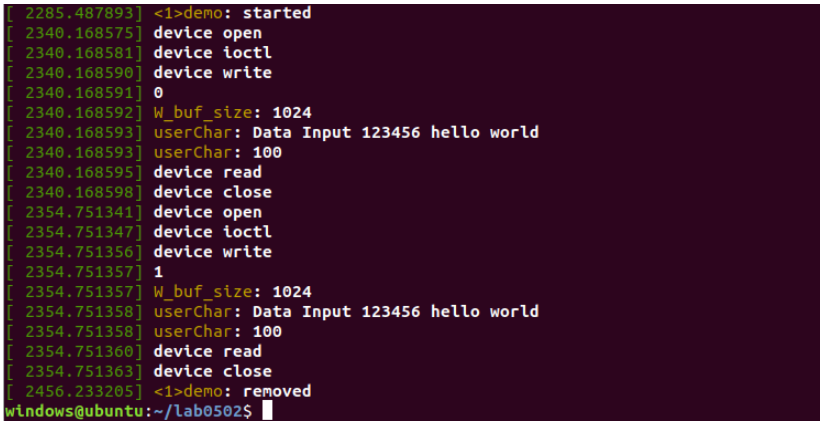
實際上的結果

- 成功編譯並載入 hellod.ko 與 demo.ko，於 TX2 上運行正常
- test.c 成功開啟 /dev/demo 並傳輸字串資料
- dmesg 顯示每次開啟、寫入、關閉皆觸發對應函式
- 模組可重複載入與移除，系統穩定，結果與預期一致

Part 1





Part 2


```
[ 2285.487893] <1>demo: started
[ 2340.168575] device open
[ 2340.168581] device ioctl
[ 2340.168590] device write
[ 2340.168591] 0
[ 2340.168592] W_buf_size: 1024
[ 2340.168593] userChar: Data Input 123456 hello world
[ 2340.168593] userChar: 100
[ 2340.168595] device read
[ 2340.168598] device close
[ 2354.751341] device open
[ 2354.751347] device ioctl
[ 2354.751356] device write
[ 2354.751357] 1
[ 2354.751357] W_buf_size: 1024
[ 2354.751358] userChar: Data Input 123456 hello world
[ 2354.751358] userChar: 100
[ 2354.751360] device read
[ 2354.751363] device close
[ 2456.233205] <1>demo: removed
windows@ubuntu:~/lab0502$
```

遇到的問題&問題如何解決

- PPT 提供的 demo.c 含有未定義變數、過時 ioctl 寫法與不相容的 file_operations，導致程式無法編譯與無法正常運作
  - 將所有過時與未定義的程式碼移除，改用新版 kernel 的 unlocked_ioctl 介面，並補上 copy_from_user 等必要函式，重新整理成簡潔且可正常編譯執行的驅動架構

3. 程式碼：

Part 1: hellod.c (tx2)

```c
#include <linux/kernel.h>
#include <linux/module.h>

static int __init tx2_hello_module_init(void)
{
   printk("Hello, Virtual Machine module is installed !\n");
   return 0;
}

static void __exit tx2_hello_module_cleanup(void)
{
   printk("Goodbye, Virtual Machine module was removed!\n");
}

module_init(tx2_hello_module_init);
module_exit(tx2_hello_module_cleanup);
MODULE_LICENSE("GPL");
```

Part 1: Makefile (tx2)

```makefile
obj-m := hellod.o

kernel_DIR := /usr/src/linux-headers-5.4.0-150-generic/

PWD := $(shell pwd)

all:
   make -C $(kernel_DIR) M=$(PWD)
clean:
   rm *.o *.ko *.mod.c
.PHONY:
   clean
```

Part 1: hellod.c (ubuntu)

```c
#include <linux/kernel.h>
#include <linux/module.h>

static int __init tx2_hello_module_init(void)
{
   printk("Hello, TX2 module is installed !\n");
   return 0;
}

static void __exit tx2_hello_module_cleanup(void)
{
   printk("Goodbye, TX2 module was removed!\n");
}

module_init(tx2_hello_module_init);
module_exit(tx2_hello_module_cleanup);
MODULE_LICENSE("GPL");
```

Part 1: Makefile (ubuntu)

```makefile
obj-m := hellod.o

kernel_DIR :=
/usr/src/linux-headers-4.9.201-tegra-ubuntu18.04_aarch64/ker
nel-4.9/

PWD := $(shell pwd)

all:
   make -C $(kernel_DIR) SUBDIRS=$(PWD)
clean:
   rm *.o *.ko *.mod.c
.PHONY:
   clean
```

Part 2: demo.c

```c
#include<linux/init.h>
#include<linux/kernel.h>
#include<linux/module.h>
#include<linux/fs.h>
#include<asm/uaccess.h>

#define MAJOR_NUM 60
#define MODULE_NAME "demo"

static int iCount = 0;
static char userChar[100];


static ssize_t drv_write(struct file *flip, const char *buf,
size_t count, loff_t *ppos){
   printk("device write\n");
   printk("%d\n", iCount);
   printk("W_buf_size: %d\n", (int)count);
   raw_copy_from_user(userChar, buf, count);
   userChar[count - 1] = 0;
   printk("userChar: %s\n", userChar);
   printk("userChar: %d\n", (int)sizeof(userChar));
   iCount++;
   return count;
}

long drv_ioctl(struct file *filp, unsigned int cmd, unsigned
long arg){
   printk("device ioctl\n");
   return 0;
}
```

```
static int drv_open(struct inode *inode, struct file *filp){
   printk("device open\n");
   return 0;
}

static int drv_release(struct inode *inode, struct file
*filp){
   printk("device close\n");
   return 0;
}

static ssize_t drv_read(struct file *flip, char *buf, size_t
count, loff_t *ppos){
   printk("device read\n");
   return count;
}

struct file_operations drv_fops =
{
   read: drv_read,
   write: drv_write,
   unlocked_ioctl: drv_ioctl,
   open: drv_open,
   release: drv_release,
};

static int demo_init(void){
   if(register_chrdev(MAJOR_NUM, "demo", &drv_fops)<0){
       printk("<1>%s: can't get major %d\n", MODULE_NAME,
MAJOR_NUM);
       return (-EBUSY);
   }

   printk("<1>%s: started\n", MODULE_NAME);
   return 0;
```

```
}

static void demo_exit(void){
    unregister_chrdev(MAJOR_NUM, "demo");
    printk("<1>%s: removed\n", MODULE_NAME);
}

module_init(demo_init);
module_exit(demo_exit);
```

Part 2: Test.c

```
#include <stdio.h>

int main()
{
    char buf[1024] = "Data Input 123456 hello world";

    FILE *fp = fopen("/dev/demo", "w+");
    if (fp == NULL) {
        printf("can't open device!\n");
        return 0;
    }

    fwrite(buf, sizeof(buf), 1, fp);

    fread(buf, sizeof(buf), 1, fp);

    fclose(fp);

    return 0;
}
```

Part 2: Makefile

```
obj-m := demo.o

kernel_DIR := /usr/src/linux-headers-5.4.0-150-generic/

PWD := $(shell pwd)

all:
   make -C $(kernel_DIR) M=$(PWD)
clean:
   rm *.o *.ko *.mod.c
.PHONY:
   clean
```

4. 本次實驗過程說明與解決方法：

本次實驗主要在 Ubuntu 與 TX2 平台上練習撰寫可載入的 Linux 驅動模組。首先建立 hellod.c 驗證驅動掛載與卸載流程，再撰寫 demo.c 實作字元裝置驅動，透過 register_chrdev() 與 file_operations 完成 open、write、read、release、ioctl 等功能。測試階段以 test.c 驗證使用者層與核心層間的資料傳遞。過程中曾遇到 Makefile 編譯與裝置節點問題，透過修正 kernel 版本與建立 /dev/demo 節點後皆順利解決。最終成功在 TX2 上完成驅動模組的載入、操作與移除，實驗結果與預期一致。

5. 分工：

學號、組員	貢獻比例	工作內容
112590450 李馥亘	25%	小組報告、程式規劃
112590452 林希芸	25%	資料查詢、文書處理
112590454 林妍蓁	25%	實驗環境準備、實驗設計
112590455 吳哲丞	25%	測試與除錯、實驗實作